Autodesk® Scaleform®

字體和文字配置

Scaleform 4.2 Scaleform C++ APIs

Maxim Shemanarev, Michael Antonov

2.2
2012 6 21

Autodesk® GAMEWARE

Copyright Notice

Autodesk® Scaleform® 4.2

Autodesk Scaleform Corporation
6305 Ivy Lane, Suite 310
Greenbelt, MD 20770, USA

www.scaleform.com

info@scaleform.com

(301) 446-3200

(301) 446-3199

1 引言：Scaleform 中的字體

Scaleform HTML
GFX/SWF
FreeType 2

Scaleform Flash® Studio SWF
Flash Studio

Flash

Scaleform GFx::Translator GFx::FontLib

FreeType2

Scaleform Flash Studio Scaleform
Flash
Scaleform

Flash Scaleform

- “Flash ” Flash
  Flash
- “ ” Flash

1.1 Flash

Flash Studio

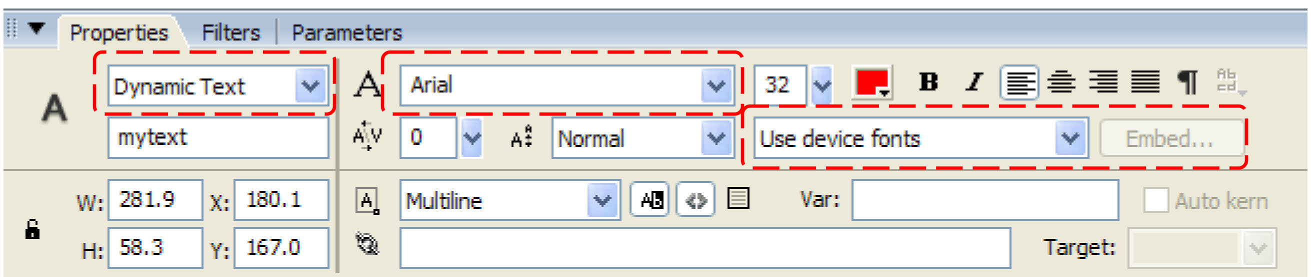
Flash
Studio

Flash
Scaleform 1 ---

1.1.1

Flash Text Tool

Flash Studio



GFx::Translator

(a) (b)
“Arial”

SWF Flash 8

-
-
-

“Use device fonts” “ ”

SWF

Flash

/

Scaleform

“ ” “ ”

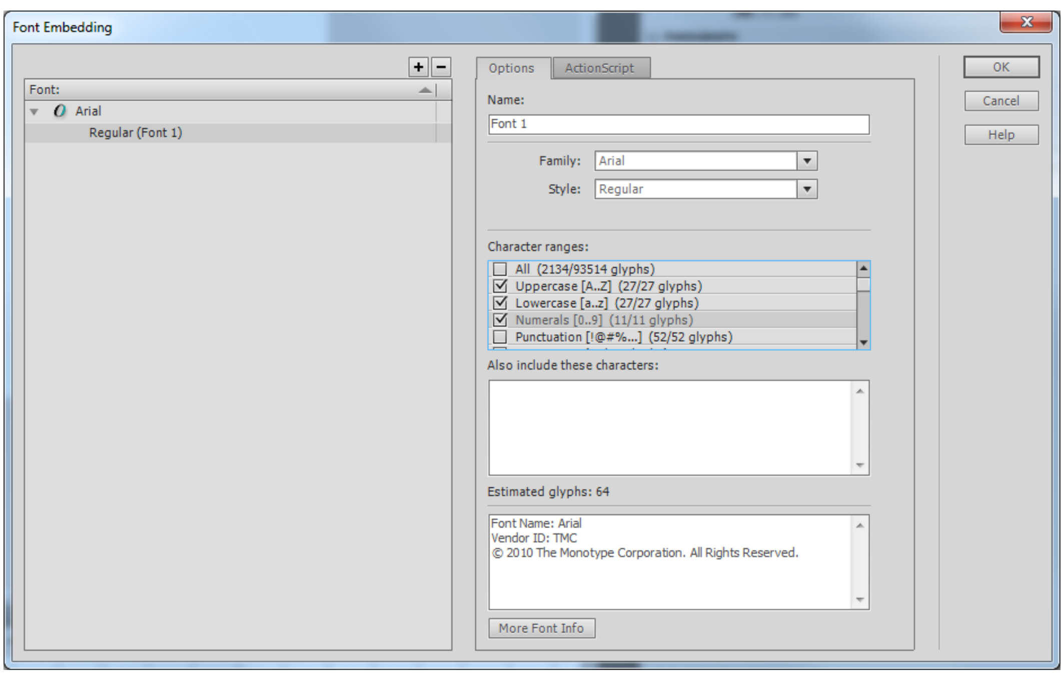
1.1.2

, TextField( ) (

,)

Scaleform Scaleform

Embed



, , (Style (
)) Arial Arial

Flash Studio “ ”

SWF/GFx

Scaleform Xbox 360
PS3 Wii Windows Mac Linux

1.1.3

Scaleform ,

	SWF	Scaleform
1	1 KB	450 K
114 – +	12 KB	480 K
596 –	70 KB	630 K
7,583 –	2,089 KB	3,500 K
18,437 –	5,131 KB	8,000 K

“Arial Unicode MS”
500 150K

1.1.4

1.

Flash

2. *Scaleform* */*

SWF/GFx

Scaleform

3.

IME

4. GFxEport (-fc) , 10%-30%,

(5.3)

Scaleform IME

1.2

Scaleform

- *1* ——
- *2* ——
- *3* —— Scaleform
- *4* -—— Scaleform

2

- Scaleform
-
- Flash SWF/GFx

Scaleform
3 “ ” Scaleform Scaleform FreeType
2 Scaleform

“ ” Scaleform
gfxexport

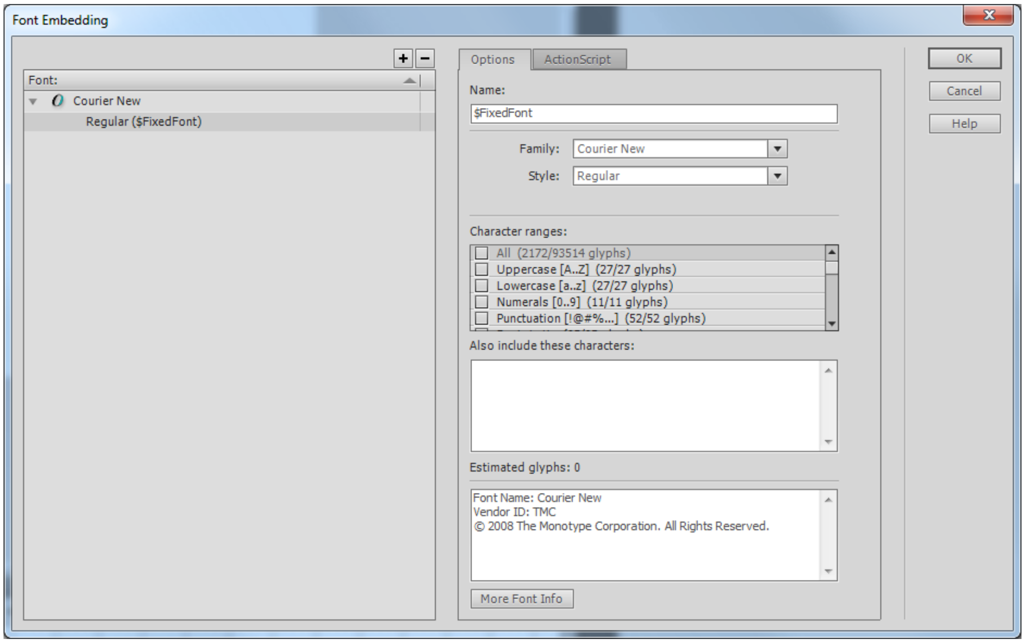
2 1

1.

2.

3.

Scaleform , ,
, , (: fonts_en.swf fonts_kr.swf)
Flash SWF



FLA

“$FixedFont”

“$ArialGame*”

“$”

“_”

“Font rendering method” “ ” Flash Studio

2.1.1

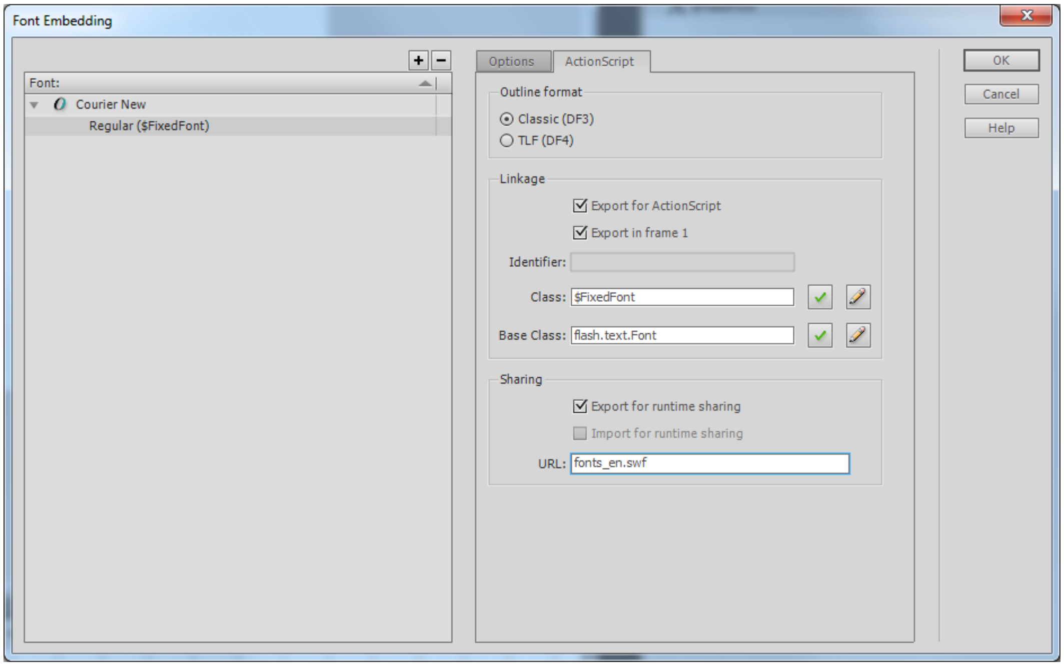
/ FLA

FLA “Copy”

FLA FLA “Paste”

Flash / ,

Properties ( ), ActionScript ,



OK ( ) ,Flash

, , Export for runtime sharing ( ) Export for ActionScript (ActionScript) Export in first frame ( ) URL SWF , ; GFx::FontLib , URL fonts_en.swf , , , GFx

Loader.SetDefaultFontLibName("fonts_en.swf");

fonconfig.txt GFxPlayer
fontlib "fonts_en.swf"

fontlib

"Export: $identifier"
/
SWF

Scaleform SWF

TextField.htmlText TextFormat
"Arial"

2.1.2

CS4 Flash Studio ,
Windows " \ \ " "
Unicode " " " 243
11920 , , ,
'gfxfontlib.swf'

, Flash CS5 , (Glyph), 'gfxfontlib.swf'
('fonts_en.swf' 'fonts_jp.swf'),

, CS4 Flash Studio( ), 'gfxfontlib.swf' (,
) , GFx 4.0

2.2 gfxfontlib.swf

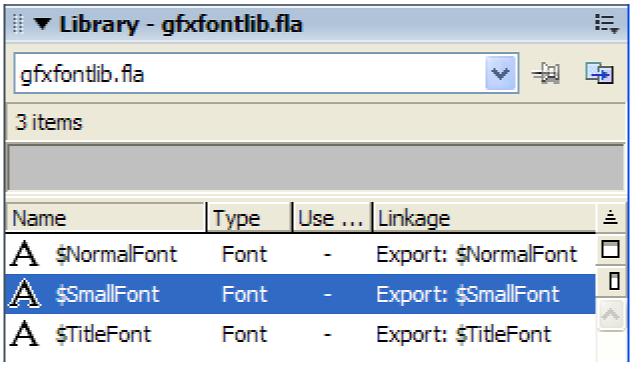
, ,

, Flash Studio FLA,

1. '$TitleFont' '$NormalFont'
2. FLA
3. 1
4.
5.
6. fonts_<lang>.swf, 'lang' (:'en' – 'ru' – 'jp' – 'cn' – 'kr' –) , FontMap </lang>

,

FLA


Library - gfxfontlib.fla
gfxfontlib.fla
3 items
Name
Type
Use ...
Linkage
$NormalFont
Font
-
Export: $NormalFont
$SmallFont
Font
-
Export: $SmallFont
$TitleFont
Font
-
Export: $TitleFont

/

3 2

1.
2.
3.

'fonts_en.swf'
GFx::FontLib GFx::FontMap

GFx::Translator

Flash Studio

3.1

Flash /
, , , , 1 fonts_en.swf ,
UI Adobe Flash
"fonts_en.swf" Scaleform

, Loader::SetDefaultFontLibName (const char* filename) C++ , (, !) ‘filename’ , ,

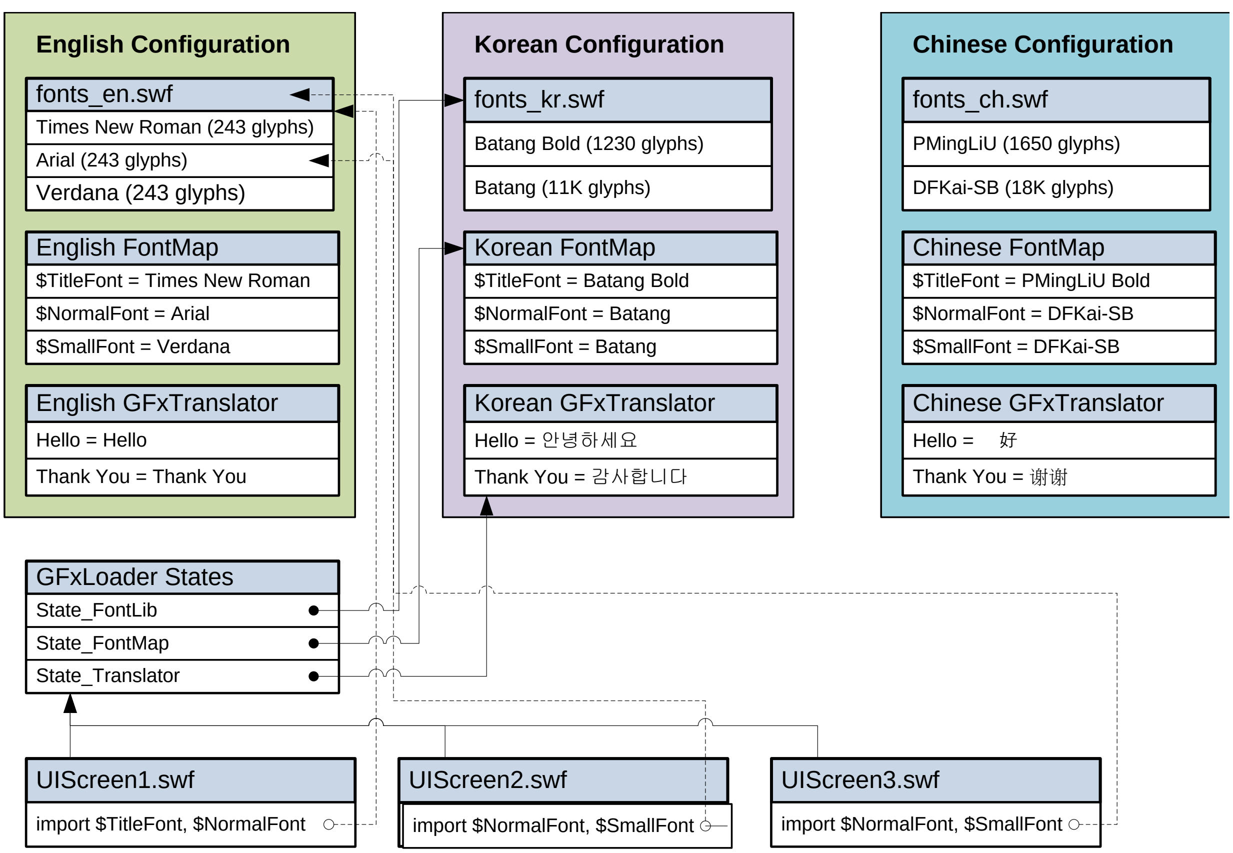
Loader.SetDefaultFontLibName(“fonts_kr.swf”);

SWF CreateInstance

1.
2.
3. “$NormalFont”

Scaleform GFx::Translator GFx::FontLib GFx::FontMap GFx::Loader GFx::Loader::CreateMovie

CreateMovie



"UIScreen1.swf"
"UIScreen3.swf" "fonts_en.swf"
Scaleform Scaleform GFx::FontLib
GFx::FontMap states :(1)
Loader::SetDefaultFontLibName() , (2) UI
GFx::FontLib

,
"fonts_kr.swf"

UIScreen

3.1.1

SWF Scaleform
GFx::FontMap GFx::Translator
Scaleform "fontconfig.txt"

1. 1 (fonts_en.swf),
2. GFx::Translator
3. GFx::FontMap
4. , fonts_kr.swf

1. GFx::Loader GFx::Translator

2. GFx::FontMap GFx::Loader MapFont
3 GFx::FontMap

3. , ,GFx::FontProvider / GFx::FontPackParams , GlyphCacheConfig

4. GFx::FontLib GFx::Loader

5. (swf), SetDefaultFontLibName(filename)

```
Loader.SetDefaultFontLibName("fonts_en.swf");
```

GFx::FontLib SWF/GFx GFx::FontLib::AddFromFile

6. GFx::Loader::CreateMovie

3.1.2 Scaleform Player

Scaleform Player "fontconfig.txt" SWF/GFX 'fontconfig.txt' /fc Ctrl+N F2 HUD

8 ASCII UTF-16

```
[FontConfig "English"]
fontlib "fonts_en.swf"
map "$TitleFont" = "Times New Roman" Normal
map "$NormalFont" = "Arial " Normal
map "$SmallFont" = "Verdana" Normal

[FontConfig "Korean"]
fontlib "fonts_kr.swf"
map "$TitleFont" = "Batang" Bold
map "$NormalFont" = "Batang" Normal
map "$SmallFont" = "Batang" Normal
tr  "Hello" = "안녕하세요"
tr  "Thank You" = "감사합니다"

[FontConfig "Chinese"]
fontlib "fonts_ch.swf"
map "$TitleFont" = "PMingLiU" Bold
map "$NormalFont" = "DFKai-SB" Normal
map "$SmallFont" = "DFKai-SB" Normal
tr  "Hello" = "    "
tr  "Thank You" = "    "
```

[FontConfig "name"]

Scaleform Player HUD

fontlib "fontfile"	SWF/GFx GFx::FontLib fontlib
map "$UIFont" = "PMingLiU"	GFx::FontMap GFx::FontMap $UIFont default “fonts_en.swf” UI
tr "Hello" = " "	

Scaleform
Scaleform
XML FontConfigParser.h/.cpp

Bin\Data\AS2\Samples\FontConfig and Bin\Data\AS3\Samples\FontConfig
Scaleform “sample.swf” “fontconfig.txt”
“sample.swf” Scaleform Player “fontconfig.txt”

3.1.3 設備字體仿真

GFx::FontMap
“Arial” “Verdana”
Scaleform Player
“Arial”
“Batang” “Batang” GFx::FontLib
“font_kr.swf”

GFx::FontLib GFx::FontProviderWin32
GFx::FontProviderFT2

- Flash Adobe Flash Player

Scaleform Flash

- “Anti-alias for animation” ActionScript TextField.antiAliasType Scaleform Player
- Scaleform Adobe Flash , , (fonts_en.swf)
- ActionScript TextFormat Flash “Arial” “$TitleFont” “fonts_en.swf” “fonts_en.swf”

3.1.4

SWF , SWF , fontconfig.txt

1. FLA, ActionScript 2.0 3.0 ‘main-app.fla’ SWF
2. FLA, ActionScript ‘fonts_en.swf’
3. ‘Library’( ) , , ‘New Font...’(...) $Font1, Family ( ) Arial , Style ( ) Regular
4. Character ranges ( ) , English( ), ‘Basic Latin’
5. ‘ActionScript’ ‘Export for ActionScript’(ActionScript) ‘Export for frame 1’(1) ‘Export for run-time sharing’( ) ‘URL’ , ‘fonts_en.swf’ ‘OK’( ) ActionScript 3.0,Flash Studio ‘A definition for this class could not be found...’(...) – , ‘OK’( )
6. 3 – 5, ‘$Font2’, ‘Family’( ) / ‘Style’( ) ,‘Family’( ) ‘Style’( ) , Flash Studio , ‘Times New Roman’
7. FLA, ActionScript ‘fonts_kr.fla’ ‘fonts_ru.fla’, , Korean( ), ‘fonts_kr.fla’

8. ‘Library’( ) , , ‘New Font...’(...) $Font1, Family ( ) (바탕), Style ( ) Regular
9. Character ranges ( ) , , ‘Korean Hangul (All)’(( ))
10. ‘ActionScript’ ‘Export for ActionScript’(ActionScript) ‘Export for frame 1’(1) ‘Export for run-time sharing’( ) ‘URL’ , ‘fonts_en.swf’ ‘OK’( ) ActionScript 3.0,Flash Studio ‘A definition for this class could not be found...’(...) – , ‘OK’( )
11. 8 – 10, ‘$Font2’, ‘Family’( ) , ,‘바탕체’
12. ‘fonts_en.swf’ ‘fonts_kr.swf’
13. main-app.fla (‘fonts_en.swf’), Library( ), ‘$Font1’ ‘$Font2’, (Ctrl-C -> ‘Copy’( ))
14. ‘main-app.fla’, Library( ), (Ctrl-V -> ‘Paste’( )) , ‘Import:’(:)
15. ‘main-app.fla’ ‘Classic Text’( ) ‘Dynamic Text’( ) ‘Family’( ) ‘$Font1*’ ‘$TEXT1’, (, ID)
16. , ‘$Font2*’, ‘$TEXT2’
17. ‘main-app.swf’,
18. , swf fontconfig.txt ‘Notepad’( ) Unicode UTF-8

```
[FontConfig "English"]
fontlib "fonts_en.swf"
map "$Font1" = "Arial"
map "$Font2" = "Times New Roman"
tr  "$TEXT1" = "This is"
tr  "$TEXT2" = "ENGLISH!"
```

```
[FontConfig "Korean"]
fontlib "fonts_kr.swf"
map "$Font1" = "Batang"
map "$Font2" = "BatangChe"
tr  "$TEXT1" = "이것"
tr  "$TEXT2" = "은 한국이다!"
```

: (바탕 바탕체), Flash SWF (Batang BatangChe) fontconfig.txt

SWF , gfxexport –fntlst<swfname> *.lst </swfname>

19. (Unicode UTF-8!) ! GFxPlayer 'main-app.swf', 'This is'( ) 'English!'(!) Ctrl-N Korean( ), fonts_kr.swf (fonts_kr.swf)

3.2

SWF/GFx "UIScreen.fla" SWF "UIScreen.swf" "UIScreen.swf"

600K

-
-
-
- IME

Flash Studio " " Ctrl + F11 "Replace strings" "manually using stage language" Flash Scaleform

4 3

Flash HTML

1.
2. SWF / GFx
3. GFx::FontLib SWF/GFx
4. GFx::FontProviderWin32

Flash

GFx::FontLib GFx::FontMap GFx::FontProvider
GFx::FontLib GFx::FontMap
SWF/GFx

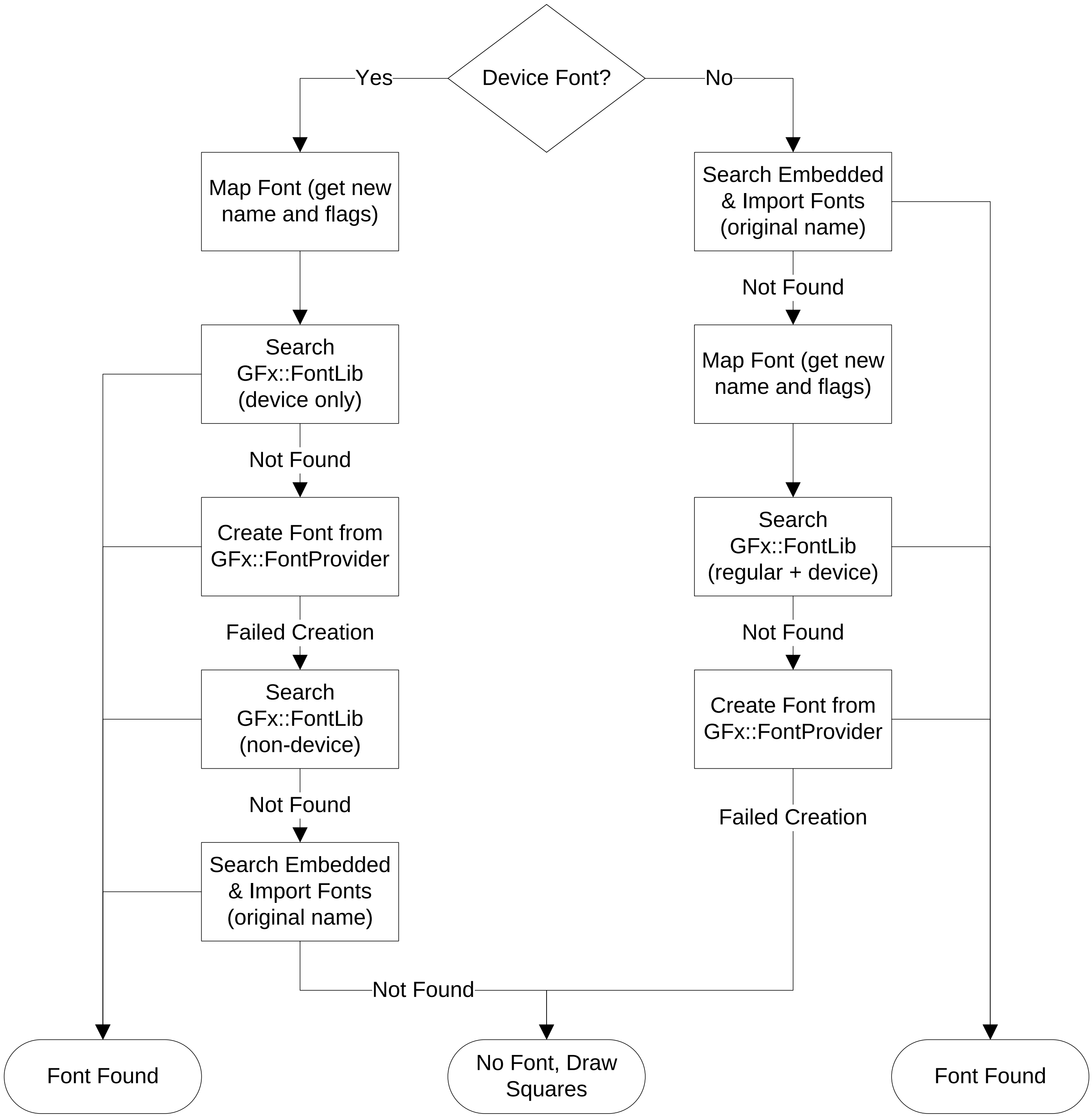
GFx::Loader::SetFontProvider
SWF SWF
Scaleform

- GFx::FontProviderWin32 – Win32 APIs
- GFx::FontProviderFT2 – David Turner FreeType-2

4.1

Scaleform
"Use device fonts"

Adobe Flash
Scaleform



GFx::FontLib GFx::FontProvider

4.2 GFx::FontMap

GFx::FontMap
GFx::FontMap

3

```
[FontConfig "Korean"]
fontlib "fonts_kr.swf"
map "$TitleFont" = "Batang" Bold
map "$NormalFont" = "Batang" Normal
map "$SmallFont" = "Batang" Normal
```

"$TitleFont "

C++

```
#include "GFx/GFx_FontLib.h"
. . .
Ptr<FontMap> pfontMap = *new FontMap;
Loader.SetFontMap(pfontMap);

pfontMap->MapFont("$TitleFont", "Batang", FontMap::MFF_Bold  scaleFactor = 1.0f);
pfontMap->MapFont("$NormalFont", "Batang", FontMap::MFF_Normal  scaleFactor = 1.0f);
pfontMap->MapFont("$SmallFont", "Batang", FontMap::MFF_Normal  scaleFactor = 1.0f);
```

; "$NormalFont" "$SmallFont" MapFont MFF_Original scaleFactor scaleFactor 1.0f

GFx::FontMap GFx::FontMap GFx::Loader::CreateMovie GFx::MovieDef

4.3 GFx::FontLib

3 ,GFx::FontLib , (1) fonts_en.swf , (2) GFx::FontLib

```
#include "GFx/GFx_FontLib.h"
. . .
Ptr<FontLib> fontLib = *new FontLib;
Loader.SetFontLib(fontLib);
fontLib->SetSubstitute(“<default_font_lib_swf_or_gfx_file>”);

Ptr<MovieDef>  m1 = *Loader.CreateMovie("<swf_or_gfx_file1>");
Ptr<MovieDef>  m2 = *Loader.CreateMovie("<swf_or_gfx_file2>");
. . .
fontLib->AddFontsFrom(m1, true);
fontLib->AddFontsFrom(m2, true);
```

AddFontsFrom AddRef M1 M2 /

GFx::FontLib

4.4 GFx::FontProviderWin32

GFx::FontProviderWin32 Win32 API GetGlyphOutline

```
#include "GFx/GFx_FontProviderWin32.h"
. . .
Ptr<FontProviderWin32> fontProvider = *new FontProviderWin32(::GetDC(0));
Loader.SetFontProvider(fontProvider);
```

GFx::FontProviderWin32 Windows Display Context DC (::GetDC(0))

4.4.1

Scaleform

font APIs (GFx::FontProviderWin32 and GFx::FontProviderFT2)

```
Font::NativeHintingRange vectorRange;
Font::NativeHintingRange rasterRange;
unsigned maxVectorHintedSize;
unsigned maxRasterHintedSize;
```

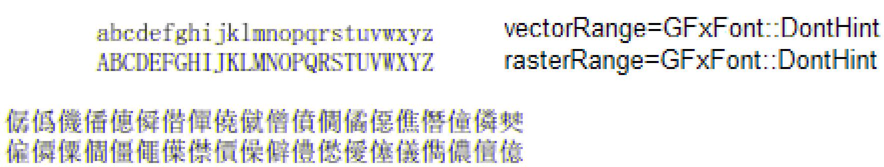
VectorRange RasterRange

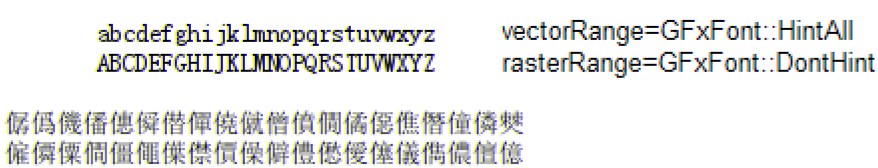
`Font::DontHint` –

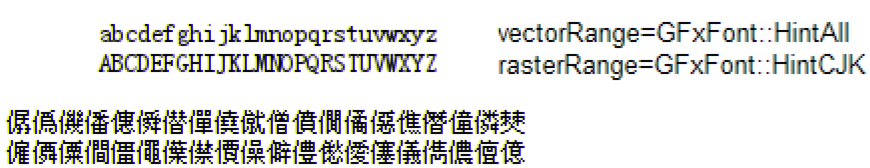
`Font::HintCJK` –

`Font::HintAll` –

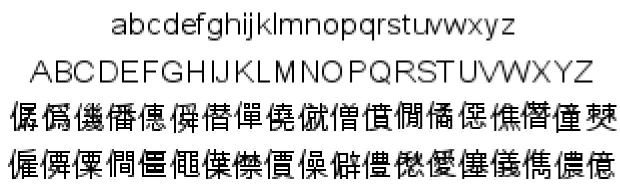
RasterRange VectorRange maxVectorHintedSize maxRasterHintedSize SimSun







"Arial Unicode MS"



Adobe Flash Scaleform

Win32 FreeType

```
vectorRange = Font::DontHint;
rasterRange = Font::HintCJK;
maxVectorHintedSize = 24;
maxRasterHintedSize = 24;
```

4.4.2

SetHinting() SetHintingAllFonts()

```
fontProvider->SetHintingAllFonts(. . .);
```

```
fontProvider->SetHinting(fontName, . . .);
```

SetHinting() SetHintingAllFonts()

```
void SetHintingAllFonts(Font::NativeHintingRange vectorRange,
                        Font::NativeHintingRange rasterRange,
                        unsigned maxVectorHintedSize=24,
                        unsigned maxRasterHintedSize=24);

void SetHinting(const char* Name,
                Font::NativeHintingRange vectorRange,
                Font::NativeHintingRange rasterRange,
                unsigned maxVectorHintedSize=24,
                unsigned maxRasterHintedSize=24);
```

Name UTF-8

GFx::FontProviderWin32
Windows API GFx::FontProviderWin32

```
void FontProviderWin32::SetRasterFormat(unsigned format, UByte* gamma=0);
```

```
GGO_BITMAP (    ),
GGO_GRAY2_BITMAP,
GGO_GRAY4_BITMAP,
GGO_GRAY8_BITMAP
```

<windows.h> GFx::FontProviderWin32.h

0…255 GGO_GRAY2_BITMAP
GGO_GRAY4_BITMAP GGO_GRAY8_BITMAP

GGO_GRAY2_BITMAP 0…4 GGO_GRAY4_BITMAP 0…16
GGO_GRAY8_BITMAP 0…64 5 17 65 0
– 255 0 255
GGO_GRAY2_BITMAP 0, 63, 127, 191, 255

4.5 GFx::FontProviderFT2

David Turner FreeType-2 library
GFx::FontProviderWin32 *GFx::FontProviderFT2*

FreeType

Windows MSVC

freetype<ver>.lib – DLL
freetype<ver>_D.lib – DLL
freetype<ver>MT.lib – DLL(CRT)
freetype<ver>MT_D.lib – DLL (CRT)

“<ver>” FreeType 2.1.9 219

freetype2/include freetype2/objs

GFx::FontProviderFT2

```
#include "GFx/GFx_FontProviderFT2.h"
. . .
Ptr<FontProviderFT2> fontProvider = *new FontProviderFT2;
<Map Font to Files or Memory>
Loader.SetFontProvider(fontProvider);
```

```
FontProviderFT2(FT_Library lib=0);
```

FreeType FreeType
FreeType Scaleform
(
FT_Done_FreeType) GFx::Loader
FreeType
fontProvider->GetFT_Library() FT_Library
FreeType GFx::FontProviderFT2
FreeType

4.5.1 FreeType

Win32 API FreeType
GFx::FontProviderFT2

```
void MapFontToFile(const char* fontName, unsigned fontFlags,
                   const char* fileName, unsigned faceIndex=0,
                   Font::NativeHintingRange vectorHintingRange = Font::DontHint,
                   Font::NativeHintingRange rasterHintingRange = xFont::HintCJK,
                   unsigned maxVectorHintedSize=24,
                   unsigned maxRasterHintedSize=24);
```

“fontName” “Times New Roman” “filename”
“C:\\WINDOWS\\Fonts\\times.ttf” “fontFlags” “Font::FF_Bold ”
“Font::FF_Italic ” “Font::FF_BoldItalic” “Font::FF_BoldItalic”
“Font::FF_Bold | Font::FF_Italic” “faceIndex” FT_New_Face
0
; Win32
FreeType

4.5.2 將字體映射到記憶體

FreeType

```
void MapFontToMemory(const char* fontName, unsigned fontFlags,
                     const char* fontData, unsigned dataSize,
                     unsigned faceIndex=0,
                     Font::NativeHintingRange vectorHintingRange = Font::DontHint,
                     Font::NativeHintingRange rasterHintingRange = Font::HintCJK,
                     unsigned maxVectorHintedSize=24,
                     unsigned maxRasterHintedSize=24);
```

```
    FILE* fd = fopen("C:\\WINDOWS\\Fonts\\times.ttf", "rb");
    if (fd) {
        fseek(fd, 0, SEEK_END);
        unsigned size = ftell(fd);
        fseek(fd, 0, SEEK_SET);
        char* font = (char*)malloc(size);
        fread(font, size, 1, fd);
        fclose(fd);
        fontProvider->MapFontToMemory("Times New Roman", 0, font, size);
    }
```

GFx::Loader

FreeType GFx

Windows FreeType

```
Ptr<FontProviderFT2> fontProvider = *new FontProviderFT2;
fontProvider->MapFontToFile("Times New Roman", 0,
                            "C:\\WINDOWS\\Fonts\\times.ttf");
fontProvider->MapFontToFile("Times New Roman", Font::FF_Bold,
                            "C:\\WINDOWS\\Fonts\\timesbd.ttf");
fontProvider->MapFontToFile("Times New Roman", Font::FF_Italic,
                            "C:\\WINDOWS\\Fonts\\timesi.ttf");
fontProvider->MapFontToFile("Times New Roman", Font::FF_BoldItalic,
                            "C:\\WINDOWS\\Fonts\\timesbi.ttf");

fontProvider->MapFontToFile("Arial", 0,
                            "C:\\WINDOWS\\Fonts\\arial.ttf");
fontProvider->MapFontToFile("Arial", Font::FF_Bold,
```

```
                            "C:\\WINDOWS\\Fonts\\arialbd.ttf");
fontProvider->MapFontToFile("Arial", Font::FF_Italic,
                            "C:\\WINDOWS\\Fonts\\ariali.ttf");
fontProvider->MapFontToFile("Arial", Font::FF_BoldItalic,
                            "C:\\WINDOWS\\Fonts\\arialbi.ttf");

fontProvider->MapFontToFile("Verdana", 0,
                            "C:\\WINDOWS\\Fonts\\verdana.ttf");
fontProvider->MapFontToFile("Verdana", Font::FF_Bold,
                            "C:\\WINDOWS\\Fonts\\verdanab.ttf");
fontProvider->MapFontToFile("Verdana",Font::FF_Italic,
                            "C:\\WINDOWS\\Fonts\\verdanai.ttf");
fontProvider->MapFontToFile("Verdana", Font::FF_BoldItalic,
                            "C:\\WINDOWS\\Fonts\\verdanaz.ttf");
. . .
Loader.SetFontProvider(fontProvider);
```

MapFontToFile()

5 4

Scaleform

1.
   /
2. gfxexport
3. GFx::FontPackParams
4.

PC Xbox 360 PS3 Nintendo Wii

GFx::FontProviderWin32 GFx::FontProviderFT2

CPU PSP PS2 gfxexport
CPU

Scaleform
“
”

5.1 配置字形緩存

Scaleform Render::GlyphCacheConfig GFx::FontPackParams
, Renderer2D ,

Scaleform

GFx 4.0

Scaleform 3.x 4.0 GFx 3.3 GFxLoader
GFxFontCacheManager , GFx 4.0 Renderer2D
GlyphCacheConfig Scaleform GFx 4.0 ,
5.2

,

GFx::FontPackParams ‘gfxexport’

```
bool Done = false;

if (Font has Static Textures with Packed Glyphs)
{
    if (GlyphSize <
        FontPackParams.TextureConfig.NominalSize * MaxRasterScale)
    {
        Draw the Glyph as a Texture using Static Cache;
        Done = true;
    }
}
else if (Dynamic Cache is Enabled AND
         GlyphSize < GlyphCacheParams.MaxSlotHeight)
{
    Draw the Glyph as a Texture using Dynamic Cache;
    Done = True;
 }


if (Not Done)
{
```

```
    Draw the Glyph as Vector Shape;
}
```

5.2

10000

, , ; ,

renderer->GetGlyphCacheConfig()->SetParams(Render::GlyphCacheParams(0));

,renderer Render::Renderer2D ,

, ,

HAL

LRU

```
Render::GlyphCacheParams gcparams;
gcparams.TextureWidth   = 1024;
gcparams.TextureHeight  = 1024;
gcparams.MaxNumTextures = 1;
gcparams.MaxSlotHeight  = 48;
gcparams.SlotPadding    = 2;
gcparams.TexUpdWidth    = 256;
gcparams.TexUpdHeight   = 512;

renderer->GetGlyphCacheConfig()->SetParams(gcparams);
```

`TextureWidth, TextureHeight` - 2

`MaxNumTextures` -

`MaxSlotHeight` -

`SlotPadding` -

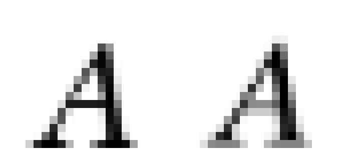
`TexUpdWidth, TexUpdWidth` – 256x512 (128K
) 256x256 128x128

“ ”
, , 1MB 4 KB
4KB 256
4K 16 100 256
4K 4K 1MB
– 256

MaxSlotHeight+2*SlotPadding
2-5 10 60-80%

500 2000
70-75

“ ” “ ”
Flash

GFX, Dynamic cache	Readability	Animation
Anti-alias for readability *Anti-alias for animation*		

“ ” Scaleform
Flash

Scaleform
(O G C Q o g e)
Scaleform

- H, E, F, T, U, V, W, X, Z
- z, x, v, w, y

Scaleform

"Warning: Font 'Arial': No hinting chars (any of 'HEFTUVWXZ' and 'zxvwy'). Auto-Hinting is disabled."

" 'Arial': ('HEFTUVWXZ' and 'zxvwy') "

Flash " "
"Zz" "Basic Latin"

5.3 – gfxexport

gfxexport SWF , GFX ,
SWF , , ,DDS
TGA –fc ,gfxeport ,
,

-fc	
-fcl <size>	, 256 , 256 25% (Flash 1024), , ,
-fcm	, 256 , 256 25% (Flash 1024), , ,

5.4 - gfxexport

–fonts gfxexport
Scaleform Player Scaleform

GFx , ,

gfxexport

-
- CPU .
- GFx::ImageCreator Render::Renderer

“ ”

‘test.swf’ ‘test.gfx’

```
gfxexport –fonts –strip_font_shapes test.swf
```

–strip_font_shapes Scaleform

gfxexport
gfxexport

-fonts	
-fns <size>	48 tri-linear mip-map
-fpp <n>	3
-fts <WxH>	256x256 ‘-fts 128' 128x128’ -fts 512x128'

-fs	
-strip_font_shapes	Scaleform
-fi <format>	<format> TGA8( ) TGA24 ( ) TGA32 DDS8 (-i option) TGA TGA8 DDS A8

: , 5.2
, ,

5.5

GFxExport

Basic Latin

Scaleform

```
Ptr<FontPackParams> packParams = *new FontPackParams();
Loader.SetFontPackParams(packParams);
```

GFxExport Scaleform “
” GFxExport

```
Loader.GetFontPackParams()->SetUseSeparateTextures(Bool flag);
Loader.GetFontPackParams()->SetGlyphCountLimit(int lim);
Loader.GetFontPackParams()->SetTextureConfig(fontPackConfig);
```

`SetUseSeparateTextures()`

`SetGlyphCountLimit()` 0

500

```
SetTextureConfig()
      FontPackParams::TextureConfig fontPackConfig;
      fontPackConfig.NominalSize   = 48;
      fontPackConfig.PadPixels     = 3;
      fontPackConfig.TextureWidth  = 1024;
      fontPackConfig.TextureHeight = 1024;
```

`NominalSize` -

"NominalSize"

NominalSize

" "

`PadPixels` — 1

`TextureWidth, TextureHeight` - 2

1) ;

2)
```
Ptr<FontPackParams> packParams = *new FontPackParams();
Loader.SetFontPackParams(packParams);
...
renderer->GetGlyphCacheConfig()->SetParams(Render::GlyphCacheParams(0));
```

3)
```
Ptr<FontPackParams> packParams = *new FontPackParams();
Loader.SetFontPackParams(packParams);
Loader.GetFontPackParams()->SetGlyphCountLimit(500);
```

500

5.6

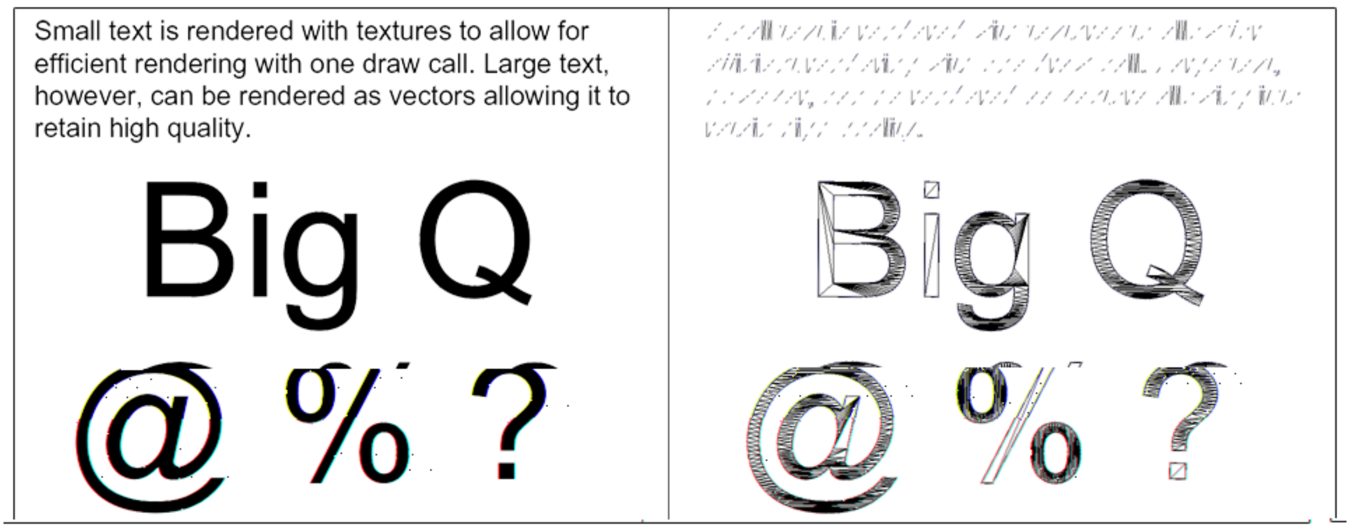
Scaleform

Flash

Scaleform

Scaleform Player

Ctrl+W



MaxSlotHeight

Render::GlyphCacheConfig::SetParams

, GlyphCacheParams::MaxRasterScale

SetMaxRasterScale

MaxRasterScale 1 1.25

1.25 48

60

MaxRasterScale -

strip_font_shapes gfxexport Scaleform

6 5

Scaleform 2.x

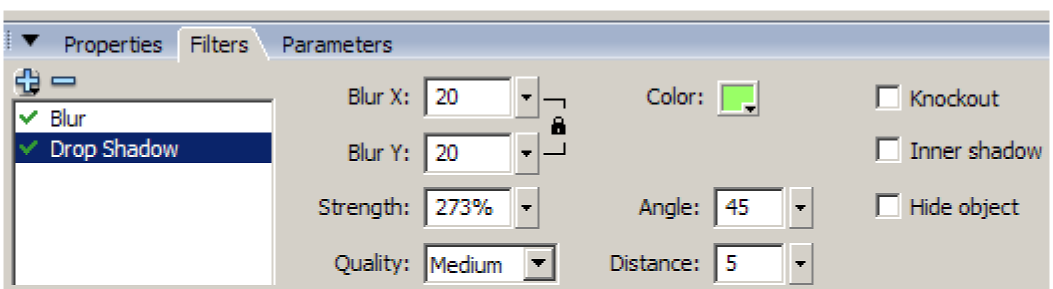
Scaleform Flash Adobe Flash ,

Flash Scaleform

Scaleform

6.1

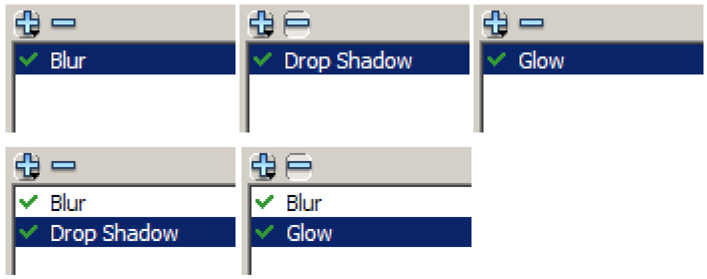
Flash Studio



Flash Scaleform Scaleform
Flash

Scaleform
Flash
Scaleform

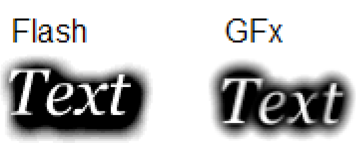
-
- “ ” “ ”
- “ ” “ ” “ ” “ ”
- “ ”
- “ ” “ ”



Scaleform Flash “ ” “ ”
“ ” “ ” “ ”
“Knockout” “Hide Object”
Scaleform “Inner Shadow” “Inner Glow”

Scaleform X Y
15.9 1590

“Knockout” Flash
Scaleform



“Knockout”

6.2

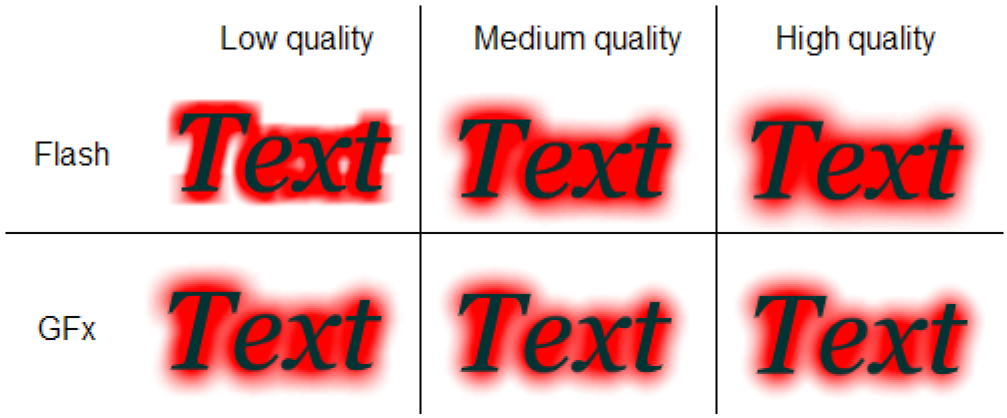
Flash “ ”
3x3 3x3
;

Flash Scaleform
Scaleform

"Quality: Low"

Scaleform

(sigma) Scaleform



Flash ; Scaleform Scaleform

http://www.ph.tn.tudelft.nl/Courses/FIP/noframes/fip-Smoothin.html.

6.3

Flash Scaleform

Scaleform

/

6.4 ActionScript

Scaleform (DropShadowFilter BlurFilter ColorMatrixFilter BevelFilter),
AS2 AS3 / , Flash